

OBJETIVOS

- Familiarizar o aluno com um SO e testes em um ambiente de máquina virtual.
- Revisitar elementos de sincronização de processos.

INFORMAÇÃO ADMINISTRATIVA

Este projeto pode ser realizado em grupos de no máximo 3 alunos. Um relatório descritivo dos procedimentos realizados e as respostas às questões presentes ao longo deste projeto devem ser entregues usando o modelo de relatório técnico-científico disponível no site da disciplina. **Cada tarefa deve ser entregue via SIGAA nos dias e prazos indicados.**

TAREFA 1: PREPARAÇÃO DO SISTEMA (de 02/12/2024 a 06/12/2024)

É recomendável que você use algum software para máquina virtual em seu computador. Para virtualização, podem ser usados softwares como o Bocks, Qemu, VirtualBox, VMware, etc. Para este projeto recomenda-se o uso do **VirtualBox**, por ser *open source* e já ter alcançado um estágio de desenvolvimento estável.

Você é encorajado a instalar o Linux caso opte por um ambiente de trabalho próprio (notebook ou máquina virtual). Adquira uma imagem de sua distribuição preferida. Verifique qual é a especificação recomendável quanto a memória e armazenamento em disco e instale o sistema.

Em caso de uso de máquina virtual, sugerem-se os seguintes passos:

- **Configure a rede da máquina virtual (demonstre que a rede funciona);**
- **Instale o ssh (cliente e servidor);**
- **Faça acesso à máquina virtual via ssh (acesso remoto). Isso pode ser feito a partir do sistema operacional hospedeiro (i.e. sistema no qual o VirtualBox está instalado);**
- **Crie um arquivo teste.txt (computador hospedeiro) e transfira-o para a máquina virtual (usando o servidor de ssh recém instalado).**
- **Crie uma conta de usuário para você (evite usar o root);**
- **Configure o bash como o *shell* padrão para o seu usuário, caso este não o seja (ou especifique qual shell está usando).**

Como o ssh já instalado em seu sistema, você pode acessá-lo de maneira mais cômoda se assim desejar (ex. usando um programa como o mucommander ou Altap Salamander¹ no sistema

¹Disponível em: <http://www.mucommander.com/> ou <https://www.altap.cz/>

operacional hospedeiro).

Inserir em tarefa apropriada no SIGAA as seguintes informações:

1. Forneça uma breve descrição do procedimento de preparação do sistema, incluindo se está usando uma máquina virtual ou SO instalado de forma nativa. Descreva o tamanho de seu disco e a quantidade de memória disponível ao S.O.
2. Caso tenha escolhido o uso de máquina virtual, apresente os passos realizados anteriormente e indique a possibilidade de conexão com a máquina virtual (host → máquina virtual) via rede.
3. Qual é o nome da aplicação (utilitário) que você usou para instalar novos softwares em seu sistema operacional? Dê exemplos.

Seja detalhista com a descrição dos procedimentos realizados até o momento.

TAREFA 2: PROCESSOS (RECAPITULANDO CONCEITOS DE PROCESSOS MEDIANTE ATIVIDADES PRÁTICAS) (de 09/12/2024 a 12/12/2024)

A seguir serão passadas uma série de atividades dirigidas cujo objetivo é recapitular conceitos fundamentais de sistemas operacionais, vistos na disciplina de SO1. **Para cada atividade, apresente o código fonte comentado e entregue este para o docente via SIGAA para análise.**

2.1. Sincronização de processos

Em SO1, muito foi discutido acerca de programação concorrente e cooperativismo entre processos. Foram apresentadas soluções para implementar a sincronização de processos, que permitem acesso a dados compartilhados de forma controlada, evitando inconsistências decorrentes da operação do escalonador de processos. Dessa forma, foram apresentados mecanismos que garantem a execução ordenada de processos cooperativos e por conseguinte, asseguram a manutenção de consistência dos dados (maiores informações nas unidades 2.6 e 2.7 de SO1).

2.1.1 Solução de Peterson

A solução de Peterson é uma solução algorítmica que permite realizar o controle de acesso a seção crítica, utilizando para isso espera ocupada e variável de guarda (Código 1). O Código 2 apresenta duas *threads* p_i e p_j , as quais operam sobre uma variável compartilhada. Pede-se a implementação da solução de Peterson para o controle de acesso a seção crítica, seguindo os moldes apresentados no Código 3. **Responda: 1) caracterize a seção crítica, levando em conta o Código 2 e a proposta de solução apresentada no Código 3; 2) como a solução de Peterson protege a seção crítica identificada?; 3) faça o tratamento de erro de modo a garantir que a *thread* foi criada adequadamente; 4) Como você faria a criação de p_i e p_j utilizando apenas uma seção de código genérica p ?; 5) Entregue o(s) código(s) resultantes dessa prática juntamente com as respostas solicitadas via SIGAA na tarefa**

apropriada.

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
        Seção crítica
    flag[i] = false;
        Restante do código
} while (true);
```

Código 1: Base para a implementação da solução de Peterson.

```
...

#define TRUE 1
#define FALSE 0

pthread_t tid[2];
int counter;

void * pi (void *arg){
    int i;

    printf("\nPi: Minha vez!!!\n");
    counter++;
    printf("\n\nPi: counter = %d\n\n", counter);
    return NULL;
}
void * pj (void *arg){
    int i;

    printf("\nPj: Minha vez!!!\n");
    counter++;
    printf("\n\nPj: counter = %d\n\n", counter);
    return NULL;
}
int main (void)
{
    pthread_create (&(tid[0]), NULL, &pi, NULL);
    pthread_create (&(tid[1]), NULL, &pj, NULL);

    pthread_join (tid[0], NULL);
    pthread_join (tid[1], NULL);

    return 0;
}
```

Código 2: Problema proposto.

```
...

#define TRUE 1
#define FALSE 0

...

void enterSC (int i)
{
    ...
}
```

```

    return;
}

void leaveSC (int i)
{
    ...

    return;
}

void * pi (void *arg){
    enterSC(0); // Entra na região crítica
    printf("\nPi: A região crítica é minha!!!\n");
    counter++;
    printf("\n\nPi: counter = %d\n\n", counter);
    leaveSC(0);
    return NULL;
}

void * pj (void *arg){
    enterSC(1); // Entra na região crítica
    printf("\nPj: A região crítica é minha!!!\n");
    counter++;
    printf("\n\nPj: counter = %d\n\n", counter);
    leaveSC(1);
    return NULL;
}

int
main (void)
{
    ...

    return 0;
}

```

Código 3: Formato da solução proposta.

2.1.2 Solução de hardware usando locks do tipo test and set

Muitos sistemas oferecem suporte de hardware para implementação de código de acesso a seção crítica (SC). Para este exercício, pede-se que o aluno utilize o a biblioteca *pthread* e a instrução `atomic_flag_test_and_set` para implementar o controle de acesso a seção crítica para as *threads* conforme criadas no Código 3. Esta solução é baseada na ideia de variável de intertravamento (*locking*) e um teste e um ajuste de valor para uma palavra de memória. **Pesquise o funcionamento desta solução e implemente o código de controle de acesso para a seção crítica apropriado. Demonstre utilizando como base o problema proposto na Seção 2.1.1.**

2.1.3 Solução de hardware usando locks do tipo mutex

As soluções apresentadas anteriormente são um pouco mais complicadas e geralmente inacessíveis a programadores de aplicação. Projetistas de SO disponibilizam ferramentas de software para resolver o problema da seção crítica, como é o caso dos *locks mutex*. No contexto de *threads*,

este é um objeto de propriedade desta, o qual permite que apenas uma única thread acesse o recurso ou SC. Utilizando a biblioteca *pthread* e a instrução *pthread_mutex_lock* / *pthread_mutex_unlock*, realize o controle de acesso a SC conforme ilustrado no Código 3. **Pesquise o funcionamento desta solução e implemente o código de controle de acesso para a seção crítica apropriado. Demonstre utilizando como base o problema proposto na Seção 2.1.1.**

2.1.4 Solução de hardware usando semáforo

Semáforo é uma ferramenta de sincronização que oferece formas mais sofisticadas (do que *mutex locks*) para que um processo sincronize suas atividades. No contexto dessa atividade, iremos utilizar apenas semáforos binários. Utilizando a biblioteca *semaphore.h* e a instrução *sem_wait/sem_post*, realize o controle de acesso a SC conforme ilustrado no Código 3. **Pesquise o funcionamento desta solução e implemente o código de controle de acesso para a seção crítica apropriado. Demonstre utilizando como base o problema proposto na Seção 2.1.1.**

TAREFA 3: DEADLOCK (de 16/12/2024 a 19/12/2024)

Deadlock é uma condição que ocorre quando dois ou mais processos estão aguardando indefinidamente por um evento que pode ser causado por apenas um dos processos em espera. Considerando, por exemplo os processos P_0 e P_1 com recursos controlados pelos semáforos S e Q, ambos inicializados em 1, pode-se dizer que um *deadlock* poderia ocorrer na condição apresentada no Pseudocódigo 1. **Sendo assim, demonstre a ocorrência de um *deadlock* utilizando como base qualquer um dos exemplos apresentados anteriormente (questões 2.1.1, 2.1.2, 2.1.3 ou 2.1.4).**

P_0	P_1
<pre>wait(S); wait(Q); ... signal(S); signal(Q);</pre>	<pre>wait(Q); wait(S); ... signal(Q); signal(S);</pre>

Pseudocódigo 1: Exemplo fictício de um deadlock.

TAREFA 4: TRATAMENTO DE DEADLOCK (de 19/12/2024 a 13/01/2025)

Considerando o algoritmo do jantar dos filósofos, implemente o cenário proposto no Pseudocódigo 2 usando *threads* e semáforos.

```

#define N      5           /* número de filósofos */
#define LEFT   (i+N-1)%N   /* número do vizinho à esquerda de i */
#define RIGHT  (i+1)%N     /* número do vizinho à direita de i */
#define THINKING 0         /* o filósofo está pensando */
#define HUNGRY  1          /* o filósofo está tentando pegar garfos */
#define EATING  2          /* o filósofo está comendo */
typedef int semaphore;     /* semáforos são um tipo especial de int */
int state[N];              /* arranjo para controlar o estado de cada um */
semaphore mutex = 1;       /* exclusão mútua para as regiões críticas */
semaphore s[N];            /* um semáforo por filósofo */

void philosopher(int i)    /* i: o número do filósofo, de 0 a N-1 */
{
    while (TRUE) {         /* repete para sempre */
        think();           /* o filósofo está pensando */
        take_forks(i);     /* pega dois garfos ou bloqueia */
        eat();             /* hummm! Espagete! */
        put_forks(i);      /* devolve os dois garfos à mesa */
    }
}

```

```

void take_forks(int i)     /* i: o número do filósofo, de 0 a N-1 */
{
    down(&mutex);          /* entra na região crítica */
    state[i] = HUNGRY;     /* registra que o filósofo está faminto */
    test(i);               /* tenta pegar dois garfos */
    up(&mutex);            /* sai da região crítica */
    down(&s[i]);            /* bloqueia se os garfos não foram pegos */
}

void put_forks(i)         /* i: o número do filósofo, de 0 a N-1 */
{
    down(&mutex);          /* entra na região crítica */
    state[i] = THINKING;   /* o filósofo acabou de comer */
    test(LEFT);            /* vê se o vizinho da esquerda pode comer agora */
    test(RIGHT);           /* vê se o vizinho da direita pode comer agora */
    up(&mutex);            /* sai da região crítica */
}

void test(i)              /* i: o número do filósofo, de 0 a N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}

```

Pseudocódigo 2: Solução algorítmica para tratamento de deadlock - jantar dos filósofos.